

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO E
ENGENHARIA DE COMPUTAÇÃO

MARIA TORRES LOUREIRO - 00577703

TRABALHO FINAL CPD

Relatório apresentado como requisito parcial para a
obtenção de conceito na Disciplina de Classificação
e Pesquisa de Dados.

Orientador: Prof. Dr. Leandro Krug Wives

Porto Alegre

2025

Sumário

	1 INTRODUÇÃO	2
	2 GUIA DE USO	
2		
	2.1 Menu de Abertura	2
	2.2 Carregar arquivo novamente	3
	2.3 Inserir novo monstro	
3		
	2.4 Pesquisa	4
	3 OS DADOS	7
	3.1 Conjunto de Dados – Monstros D&D 5ª Edição	7
	3.2 Coleta de Dados Brutos	7
	3.3 Estruturas de dados	
8		
	4 FUNÇÕES DO PROGRAMA	11
	4.1 monsters_D&D5e.py	
12		
	4.2 tress.py	23
	4.3 monsters.py	26
	5 CONCLUSÃO	
27		

1. INTRODUÇÃO

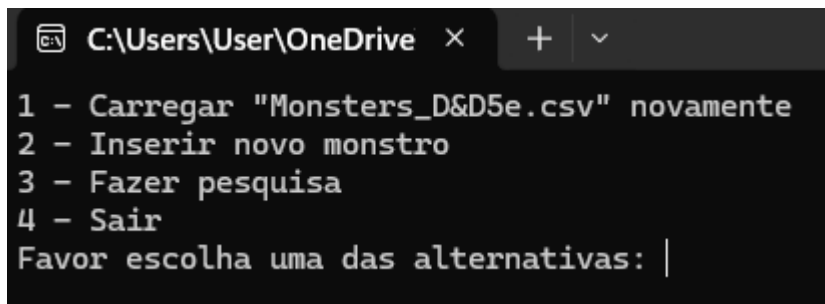
Este trabalho é uma das atividades desenvolvidas na cadeira de Classificação e Pesquisa de Dados e tem por objetivo aprofundar os alunos em conceitos relacionados a armazenamento e manipulação de dados em arquivos, indexação, pesquisa e ordenação de dados. Ele consiste em uma aplicação que permite ao usuário catalogar e pesquisar monstros da 5ª Edição do RPG Dungeons & Dragons e foi desenvolvida pela aluna Maria Torres Loureiro

2. GUIA DE USO

Este capítulo oferece um guia de uso pro programa, especificando as funcionalidades do programa por meio de explicações simplificadas do fluxo do mesmo pelo ponto de vista do usuário. Explicações mais detalhadas sobre a implementação são feitas no terceiro capítulo.

2.1 Menu de Abertura

A primeira funcionalidade do programa é o menu de abertura, responsável por mostrar as funcionalidades do programa ao usuário e coletar e validar sua resposta



```
C:\Users\User\OneDrive x + v
1 - Carregar "Monsters_D&D5e.csv" novamente
2 - Inserir novo monstro
3 - Fazer pesquisa
4 - Sair
Favor escolha uma das alternativas: |
```

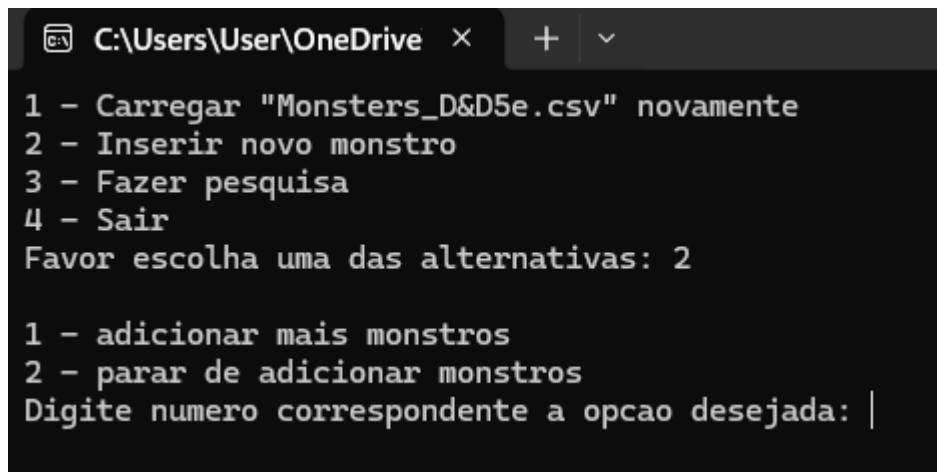
Print do menu no executável do programa

2.2 Carregar arquivo novamente

Caso a escolha for 1, ele irá carregar novamente “Monsters_D&D5e.vsc” e criar novos arquivos de índice correspondentes.

2.3 Inserir novo monstro

Caso a escolha for 2, ele irá abrir outro menu.



```
C:\Users\User\OneDrive x + v
1 - Carregar "Monsters_D&D5e.csv" novamente
2 - Inserir novo monstro
3 - Fazer pesquisa
4 - Sair
Favor escolha uma das alternativas: 2

1 - adicionar mais monstros
2 - parar de adicionar monstros
Digite numero correspondente a opcao desejada: |
```

Print do menu para adicionar monstros

Caso a resposta for 1 ele fará diversas perguntas ao usuário, que pode continuar adicionando monstros se quiser, já que ao final do cadastro de um monstro ele volta ao menu anterior

```
C:\Users\UserOneDrive x + v
Digite o nome do monstro que deseja inserir: Orgruh'rks

1 - Tiny
2 - Small
3 - Medium
4 - Large
5 - Huge
6 - Gargantuan
Qual o tamanho do monstro? 4

1 - Aberration
2 - Beast
3 - Celestial
4 - Construct
5 - Dragon
6 - Elemental
7 - Fey
8 - Fiend
9 - Giant
10 - Humanoid
11 - Monstrosity
12 - Ooze
13 - Plant
14 - Undead
Qual o tipo do monstro? 11

1 - LG
2 - NG
3 - CG
4 - LN
5 - TN
6 - CN
7 - LE
8 - NE
9 - CE
10 - ANY
Qual o alinhamento do monstro? 8
Digite o CA do monstro: 16
Digite o HP do monstro: 80
Digite o CR do monstro: 9

1 - Monster Manual
2 - Tomb of Annihilation
3 - Mordenkainen's Tome of Foes
4 - Hoard of the Dragon Queen
5 - Curse of Strahd
6 - Tales from the Yawning Portal
7 - Volo's Guide to Monsters
8 - Dungeon Master's Guide
9 - Out of the Abyss
10 - Princes of the Apocalypse
11 - Rise of Tiamat
12 - Storm King's Thunder
13 - Xanathar's Guide to Everything
14 - The Turtle Package
15 - Homebrew
De qual livro veio esse monstro? 15
```

Prints do executável ao longo do processo de cadastro de monstros

2.4 Pesquisa

Caso a escolha for 3, o programa mostrará a seguinte tela e irá pegar o input do usuário, que determinará a partir de qual atributo a pesquisa será realizada

```
1 - Nome
2 - Tamanho
3 - Tipo
4 - Alinhamento
5 - CA
6 - HP
7 - CR
8 - Fonte
Digite o numero do tipo de pesquisa que deseja realizar: 8
```

Prints do executável depois de escolher pesquisar monstro

Com base na escolha do usuário, o programa mostra as diferentes opções em cada categoria e depois de obter uma resposta e validá-la, mostra os monstros que se encaixam nessa categoria, estatísticas sobre quantas vezes ela aparece ao todo e em porcentagem e pergunta ao usuário se ele deseja continuar pesquisando monstros.

```
1 - Monster Manual
2 - Tomb of Annihilation
3 - Mordenkainen's Tome of Foes
4 - Hoard of the Dragon Queen
5 - Curse of Strahd
6 - Tales from the Yawning Portal
7 - Volo's Guide to Monsters
8 - Dungeon Master's Guide
9 - Out of the Abyss
10 - Princes of the Apocalypse
11 - Rise of Tiamat
12 - Storm King's Thunder
13 - Xanathar's Guide to Everything
14 - The Turtle Package
15 - Homebrew
De qual livro veio esse monstro? 15
```

Prints do executável pedindo input do usuário sobre as diferentes opções na categoria escolhida, nesse caso, fontes

```
Nome: Zombie, Strahd
Tamanho: Medium
Tipo: Undead
Alinhamento: U
CA: 8
HP: 30
CR: 1.00
Fonte: Curse of Strahd

Total de monstros com fontes curse of strahd: 9
Porcentagem de monstros com fontes curse of strahd: 1.12%

Deseja realizar outra pesquisa? (s/n): |
```

Prints do executável mostrando os monstros resultantes da pesquisa, mostrando as estatísticas e perguntando ao usuário se ele deseja seguir com a pesquisa

É importante notar que, no caso dos atributos numéricos, como CA, HP e CR também é possibilitado ao usuário ordenar as opções em ordem crescente ou decrescente, como pode se observar nas imagens abaixo:

```

1 - Nome
2 - Tamanho
3 - Tipo
4 - Alinhamento
5 - CA
6 - HP
7 - CR
8 - Fonte
Digite o numero do tipo de pesquisa que deseja realizar: 5
Aperte 1 para visualizar as opcoes em ordem crescente e 2 para decrescente: 1

```

```

1 - 5.0
2 - 6.0
3 - 7.0
4 - 8.0
5 - 9.0
6 - 10.0
7 - 11.0
8 - 12.0
9 - 13.0
10 - 14.0
11 - 15.0
12 - 16.0
13 - 17.0
14 - 18.0
15 - 19.0
16 - 20.0
17 - 21.0
18 - 22.0
19 - 25.0
20 - TEMP

```

Escolha o numero do lado do CA do monstro desejado: 20

opções de CR em ordem crescente

```

1 - Nome
2 - Tamanho
3 - Tipo
4 - Alinhamento
5 - CA
6 - HP
7 - CR
8 - Fonte
Digite o numero do tipo de pesquisa que deseja realizar: 5
Aperte 1 para visualizar as opcoes em ordem crescente e 2 para decrescente: 2

```

```

1 - 25.0
2 - 22.0
3 - 21.0
4 - 20.0
5 - 19.0
6 - 18.0
7 - 17.0
8 - 16.0
9 - 15.0
10 - 14.0
11 - 13.0
12 - 12.0
13 - 11.0
14 - 10.0
15 - 9.0
16 - 8.0
17 - 7.0
18 - 6.0
19 - 5.0
20 - TEMP

```

Escolha o numero do lado do CA do monstro desejado: 1

opções de CR em ordem decrescente

3. OS DADOS

Este capítulo tem o objetivo de descrever qual conjunto de dados selecionado e explicar a estrutura de dados utilizada para sua manipulação no programa elaborado

3.1 Conjunto de Dados – Monstros D&D 5ª Edição

Ao pensar em um conjunto de dados para a elaboração deste trabalho imediatamente pensei nos monstros da quinta edição do RPG Dungeons & Dragons. Tal escolha foi motivada por dois motivos específicos: o primeiro é pessoal, relacionado a uma possível aplicação futura do programa desenvolvido no dia-a-dia do meu grupo de amigos. O segundo, mais importante para o trabalho, é que esses monstros existem em grande quantidade (mais de 800) e tem características organizadas em categorias, que poderiam facilmente servir de base para a elaboração dos arquivos de índices.

Cada monstro possui diversas características. As mais importantes, que serviram como base para a elaboração do trabalho foram Nome, Tamanho, que representa as dimensões da criatura em categorias como “Tiny” e “Huge”, Tipo, como “Humanoid” e “Fiend”, Alinhamento, que caracteriza seu comportamento em duas escalas, bom/mal e leal/caótico, Classe de Armadura (em inglês “AC” e em português “CA”), que representa o quão difícil é acertar uma criatura, Pontos de Vida (HP), que indica quanta vida o ser tem, Nível de Desafio(CR), que comunica o quão difícil é enfrentá-lo e, por fim, Fonte, ou em qual dos livros do sistema de RPG aquele monstro foi lançado

3.2 Coleta de Dados Brutos

Os dados necessários para a elaboração deste trabalho foram retirados de uma tabela excel(<https://docs.google.com/spreadsheetsd/16ajgJpvUI0wYcSU7kjHutw8oB6Zv6eIAo5JtI6PLvu8/edit?gid=776794522#gid=776794522>) divulgada em um post na rede social Reddit a

(https://www.reddit.com/r/UnearthedArcana/comments/8zvr6s/the_great_dd5e_monster_spreadsheet/?rdt=43465). Contudo, é preciso destacar que depois de baixar a planilha no formato .csvs eu abri o arquivo texto e ajustei algumas inconsistências no que se diz respeito à formatação de Tamanho (deixei todos eles com a primeira letra maiúscula, como é o padrão de representação) e do Tipo (cada criatura pode ter uma categoria seguida de subcategorias entre parênteses, mas não pode haver nenhuma vírgula). Tais ajustes se fizeram necessários para lidar com inconsistências que se apresentaram ao longo do desenvolvimento do trabalho. Ambas poderiam ter sido solucionadas por meio de ajustes no código, mas acabei optando por ajustar diretamente na tabela para facilitar a elaboração do programa.

3.3 Estruturas de Dados

Para facilitar a manipulação dos dados dentro do programa elaborei três estruturas de dados específicas para lidar com os dados referentes aos monstros: `Monster`, `Monster_Name` e `Monster_Attribute`.

`Monster` tem 7 campos, cada um para um dos atributos citados anteriormente (nome, tamanho, tipo, alinhamento, AC, HP, CR e fonte) e é a principal estrutura de dados utilizada, permitindo a coleta das informações do arquivo .csvs, adição de novos monstros e parcialmente responsável pela pesquisa. Já as demais estruturas, `Monster_Name` e `Monster_Attribute` têm dois campos, o atributo em si e um valor de `index`, representando em qual linha do arquivo dos monstros contém o item que se encaixa nesse atributo.

Foi necessária a elaboração de duas estruturas de dados uma vez que nomes tiveram que ser armazenados em uma lista ao invés de em uma árvore. Tal necessidade se deu porque queria que a procura por nomes funcionasse de duas formas diferentes de forma simultânea. A primeira é a mais direta, onde um usuário digita o nome do monstro e recebe de volta o

item correspondente. A segunda funciona de maneira análoga a “tags”, e foi pensada já que muitas criaturas têm nomes parecidos. Os dragões, por exemplo, são separados por cor e idade, com a distinção sendo feita somente no nome. Logo, se um usuário quisesse todos os dragões vermelhos, eu queria que o programa devolvesse todos as instâncias dessas criaturas, com todas as idades possíveis.

Embora tenha tentado armazenar os nomes em árvores e mesmo assim ter essa pesquisa mais abrangente, não tive sucesso. Logo, resolvi manter os demais atributos em árvores, que foram implementadas exigindo que os índices de acesso fossem armazenados em listas, uma vez que cada nodo da árvore representa uma categoria de atributo e os índices representam os monstros que pertencem aquela categoria

```
1 class Monster:
2     def __init__(self, name, size, type, alignment, AC, HP, CR, source):
3         self.name = name[:32]
4         self.size = size[:10]
5         self.type = type[:40]
6         self.alignment = alignment[:11]
7         self.AC = AC
8         self.HP = HP
9         self.CR = CR
10        self.source = source[:30]
11
12 class Monster_Name:
13     def __init__(self, attribute, index):
14         self.attribute = attribute[:32]
15         self.index = index
16
17 class Monster_Attribute:
18     def __init__(self, attribute, index):
19         self.attribute = attribute
20         self.indexes = [index]
21
22     def get_index_string(self):
23         return ",".join(map(str, self.indexes))
```

Print das estruturas de dados citadas no VS Code

Além disso, para fazer uso de árvores para criar os arquivos de índices, possibilitando facilitar a pesquisa, foram criadas uma estrutura referente a árvores Trie (usadas para tamanho, tipo, alinhamento e fontes) e BST (usadas para CA, HP e CR)

É importante notar que os atributos ligados a BST também podem ser strings, pois alguns monstros podem ter valores como “TEMP” nesses campos. Contudo, esse comportamento é uma exceção.

```
11 # definicao da classe TrieNode, representando um no de uma Trie (arvore de prefixos)
12 class TrieNode:
13     def __init__(self) -> None:
14         """
15         Inicializa um no de Trie.
16         - `children`: dicionario de filhos do no, onde a chave e o caractere e o valor e o próximo no.
17         - `is_end_of_word`: booleano que indica se este no e o fim de uma palavra.
18         - `indexes`: uma string que armazena os indices dos monstros associados ao atributo.
19         """
20         # filhos do no
21         self.children = {}
22         # indica se e o fim de uma palavra
23         self.is_end_of_word = False
24         # indices dos monstros
25         self.indexes = ""
26
27 # definicao da classe Trie, que representa a estrutura da arvore Trie para busca eficiente
28 class Trie:
29     def __init__(self) -> None:
30         """
31         Inicializa a Trie com a raiz como um TrieNode vazio.
32         """
33         self.root = TrieNode()
```

Print da estrutura referente as árvores Trie no VS Code

```
95 # Definição da classe BSTNode, representando um no de uma Árvore de Busca Binária (BST).
96 class BSTNode:
97     def __init__(self, key):
98         """
99         Inicializa um no de árvore binária com um chave (atributo).
100
101         Args:
102         |   key (Monster_Attribute): O atributo que será armazenado no nó.
103         """
104         self.key = key # o atributo do monstro
105         self.left = None # no filho a esquerda
106         self.right = None # no filho a direita
107
108 # Definicao da classe BST, representando uma arvore de Busca Binária (Binary Search Tree).
109 class BST:
110     def __init__(self):
111         """
112         Inicializa a arvore binaria com a raiz como `None`.
113         """
114         self.root = None
```

A árvore Trie foi escolhida para os atributos citados pois eles são caracterizados por strings, o que bem utilizado nas Tries uma vez que cada letra de cada chave (nesse caso os atributos) vira um nodo. No caso dessa aplicação, o nodo final de um atributo tem uma lista de índices ligando-a aos monstros correspondentes

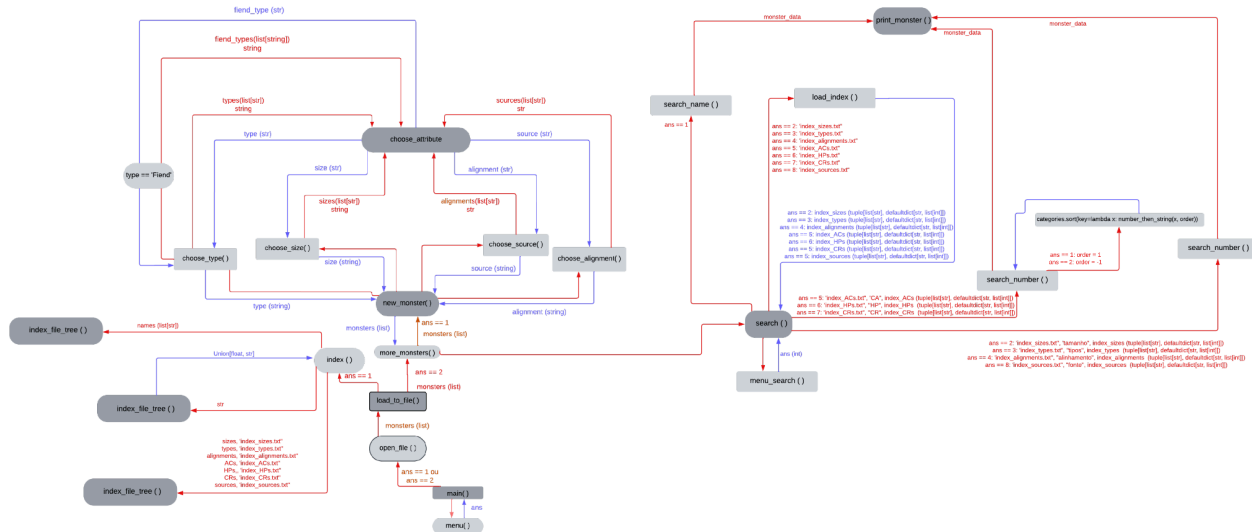
4. FUNÇÕES DO PROGRAMA

Este capítulo fala mais a fundo sobre as funções no arquivo “monsters_D&D5e” e as funções nos arquivos “monsters.py ” e “trees.py”, que contém as estruturas de dados. Para a primeira, apresenta o fluxo do programa no sentido funcional por meio de um diagrama, e para todas as três e cita cada função, uma explicação do seu funcionamento. Por fim, comenta sobre a escolha da linguagem de programação, sobre os usos de estruturas primitivas, sobre a utilização de arquivos texto ao invés de arquivos binários e sobre os módulos utilizados.

Funções são explicadas em mais detalhes nos comentários dos arquivos de código-fonte.

4.1 monsters_D&D5e.py

4.1.1 Diagrama de fluxo



4.1.2 Funções

Os textos abaixo foram tiradas da documentação de cada função no código fonte do programa

1. menu():

Exibe as opções de funcionalidades do programa e retorna a escolha do usuário.

Exibe um menu com as seguintes opções:

1. Carregar o arquivo "Monsters_D&D5e.csv" novamente.
2. Inserir um novo monstro.
3. Fazer uma pesquisa.
4. Sair do programa.

Valida a entrada do usuário para garantir que a escolha seja válida.

Returns: int: A opção escolhida pelo usuário (1, 2, 3 ou 4).

2. open_file

Leve um arquivo .csv e converta cada linha em um objeto do tipo `Monster`.

Esta função abre o arquivo especificado, extrai os dados necessários de cada linha com base em colunas predefinidas, e utiliza esses dados para criar uma lista de objetos `Monster`.

Args: file_name (str): O nome do arquivo .csv a ser aberto.

Returns: list[Monster]: Uma lista contendo instâncias do tipo `Monster`, com cada instância representando uma linha do arquivo.

Notas:

- As colunas selecionadas do arquivo .csv são baseadas no índice definido em `desired\_info`.
- As colunas consideradas são:

0. Nome do monstro

1. Tamanho

2. Tipo

3. Alinhamento

4. Classe de Armadura (AC)

5. Pontos de Vida (HP)

18. Nível de Desafio (CR)

20. Fonte

- Caso alguma coluna especificada não exista em uma linha, será substituída por uma string vazia.

3. load_to_file:

Esta função recebe uma lista de instâncias do tipo `Monster` e escreve seus atributos no arquivo "monsters.txt", onde cada linha do arquivo representa um monstro.

Args: monsters (list[Monster]): Lista de objetos do tipo `Monster` a serem salvos no arquivo.

Notas:

- Cada linha do arquivo resultante contém os atributos de um monstro, separados por vírgulas.

- Os atributos escritos no arquivo são:

- name: Nome do monstro
- size: Tamanho
- type: Tipo
- alignment: Alinhamento
- AC: Classe de Armadura
- HP: Pontos de Vida
- CR: Nível de Desafio
- source: Fonte

4. choose_attribute:

Esta função apresenta uma lista de opções de atributos para o usuário, exibe uma mensagem personalizada e valida a escolha do usuário até que uma entrada válida seja fornecida.

Args: attributes (list[str]): Lista de opções de atributos disponíveis para o usuário escolher.

message (str): Mensagem personalizada pedindo ao usuário para fazer uma escolha.

Returns: str: A opção de atributo escolhida pelo usuário.

Raises: ValueError: Se o número inserido pelo usuário não estiver entre as opções válidas.

5. choose_size:

Solicita ao usuário que escolha o tamanho de um monstro entre várias opções predefinidas. A função cria uma lista de opções de tamanhos de monstros e passa essa lista para a função `choose\_attribute`, onde o usuário escolhe uma das opções. A função retorna o tamanho escolhido.

Returns: str: O tamanho do monstro escolhido pelo usuário.

6. choose_type:

Solicita ao usuário que escolha o tipo de um monstro entre várias opções predefinidas. A função cria uma lista de opções de tamanhos de monstros e passa essa lista para a função `choose\_attribute`, onde o usuário escolhe uma das opções. Se o tipo escolhido for "Fiend", A função solicita uma escolha adicional para a categoria de "Fiend" entre três opções. A função retorna o tipo ou a categoria de "Fiend" escolhida.

Returns: str: O tipo ou a categoria de "Fiend" do monstro escolhido pelo usuário.

7. choose_alignment:

Solicita ao usuário que escolha o alinhamento de um monstro entre várias opções predefinidas. A função cria uma lista de opções de alinhamento para o monstro e passa essa lista para a função `choose\_attribute`, onde o usuário escolhe o alinhamento do monstro. O alinhamento pode ser um dos valores tradicionais de D&D ou "ANY" para qualquer alinhamento.

Returns: str: O alinhamento do monstro escolhido pelo usuário.

8. `choose_source`:

Solicita ao usuário que escolha a fonte de um monstro entre várias opções predefinidas. A função cria uma lista de fontes de onde o monstro pode ter vindo, incluindo livros oficiais de Dungeons & Dragons, como "Monster Manual" e "Tales from the Yawning Portal", e também a opção "Homebrew" para monstros criados pelo usuário ou fora dos livros oficiais. A lista de fontes é passada para a função `'choose_attribute'`, onde o usuário faz a escolha.

Returns: str: O nome da fonte de onde o monstro foi originado, conforme escolhido pelo usuário.

9. `new_monster`:

Solicita ao usuário informações sobre um novo monstro, cria um objeto `Monster` e o adiciona a uma lista existente. A função pede ao usuário o nome, tamanho, tipo, alinhamento, CA (Classe de Armadura), HP (Pontos de Vida), CR (Nível de Desafio) e a fonte de onde o monstro foi originado. Antes de adicionar o monstro a lista, a função verifica se o nome já está presente na lista de monstros. Se o nome já existir, a função informa o usuário e retorna a lista original.

Args: `monsters (list[Monster])`: A lista de monstros a qual o novo monstro será adicionado.

Returns: `list[Monster]`: A lista de monstros atualizada, incluindo o novo monstro, caso o nome não esteja duplicado.

10. `more_monsters`:

Oferece opções ao usuário para adicionar mais monstros a lista, seja carregando mais monstros de uma planilha CSV ou inserindo manualmente novos monstros. Após a interação

do usuário, atualiza o arquivo "monsters.txt" com a lista final de monstros. A função permite adicionar monstros manualmente, solicitando informações do usuário sobre o novo monstro. Parar de adicionar monstros. Quando o usuário decide parar, o arquivo "monsters.txt" é sobrescrito com a lista final de monstros.

Args: monsters (list[Monster]): A lista de monstros que será modificada durante a execução da função.

11. index_file_tree:

A função percorre a árvore de forma ordenada e para cada valor de atributo presente nos nós, ela escreve em uma linha do arquivo o valor do atributo seguido dos índices dos monstros que possuem esse valor de atributo no arquivo "monsters.txt". Caso um valor de atributo tenha múltiplos índices associados, todos os índices são listados separados por vírgulas.

Args: tree (BST): A árvore binária de busca que contém os valores dos atributos e os índices dos monstros.

file_name (str): O nome do arquivo onde os índices serão gravados.

12. index_file:

Recebe uma lista de objetos `Monster\_Attribute` e escreve o nome e o Índice de acesso de cada objeto em um arquivo de Índices. A função percorre a lista de `Monster\_Attribute`, extraindo o atributo `attribute` (nome) e o `index`, e grava essas informações no arquivo de texto "index_names.txt", onde cada linha contém o nome e o índice correspondente.

Args: names (list of Monster_Attribute): A lista de objetos `Monster\_Attribute` que contém o nome (atributo `attribute`) e o índice (atributo `index`) a ser gravado no arquivo.

13. to_number:

A função tenta converter o valor passado para um número de ponto flutuante. Se a conversão for bem-sucedida, o valor convertido é retornado. Caso contrário, o valor original (como string) retorna.

Args: value (str): O valor a ser convertido para número.

Returns: Union[float, str]: O valor convertido para 'float' se possível, ou o valor original caso contrário.

14. index:

Abre o arquivo "monsters.txt", que contém um monstro por linha, coleta os dados de cada atributo, os organiza em uma árvore (ou lista no caso dos nomes) e cria arquivos de índices que contém os diferentes valores de cada atributo e o número da linha dos monstros correspondentes no arquivo original.

Os índices gerados são:

- 'index_names.txt': Contém os nomes dos monstros e os índices das linhas.
- 'index_sizes.txt': Contém os tamanhos dos monstros e os índices das linhas.
- 'index_types.txt': Contém os tipos dos monstros e os índices das linhas.
- 'index_alignments.txt': Contém os alinhamentos dos monstros e os índices das linhas.
- 'index_ACs.txt': Contém os valores de AC (Classe de Armadura) dos monstros e os índices das linhas.
- 'index_HPs.txt': Contém os valores de HP (Pontos de Vida) dos monstros e os índices das linhas.

- ``index_CRs.txt``: Contém os valores de CR (Nível de Desafio) dos monstros e os índices das linhas.

- ``index_sources.txt``: Contém as fontes dos monstros e os índices das linhas.

15. `number_then_string`:

A função tenta converter o valor passado para float. Se a conversão for bem-sucedida, a função retorna uma tupla com a flag ``0`` e o valor convertido para float. Se a conversão falha, a função retorna uma tupla com a flag ``1`` e o valor original como string.

Args: `value (Union[str, float])`: O valor a ser processado. Pode ser uma string ou um número.

`order (int)`: flag que indica se os números devem ser ordenados em ordem crescente ou decrescente

Returns: `Tuple[int, Union[float, str]]`:

- Se a conversão para float for bem-sucedida, retorna uma tupla ``(0, float)``.
- Se a conversão falhar, retorna uma tupla ``(1, str)``, mantendo o valor original como string.

16. `print_monster`:

Recebe um dado de monstro no formato de string, cria um objeto ``Monster`` com os atributos correspondentes e imprime suas informações na tela. A função divide os dados do monstro em 8 atributos e os imprime de forma legível, incluindo nome, tamanho, tipo, alinhamento, CA, HP, CR e fonte.

Args: `monster_data (str)`: Uma string contendo os dados de um monstro, separados por vírgulas. Espera-se que a string tenha a seguinte estrutura: `'nome,tamanho,tipo,alinhamento,CA,HP,CR,fonte'`.

17. `load_index`:

Abre um arquivo de índice que contém linhas no formato 'atributo, lista de índices', onde a lista de índices indica as linhas do arquivo 'monsters.txt' que possuem o valor de atributo correspondente. Cria um dicionário onde cada valor de atributo está relacionado aos números das linhas dos monstros correspondentes no arquivo 'monsters.txt'.

Args: file (str): O nome do arquivo de índice a ser carregado. O arquivo deve ter o formato de "atributo, lista de índices", onde os índices são separados por vírgulas.

Returns: Tuple[list[str], defaultdict[str, list[int]]]:

- A lista de categorias de atributos encontradas no arquivo.
- Um dicionário onde cada chave é um valor de atributo e o valor é uma lista de índices das linhas de monstros no arquivo 'monsters.txt' que possuem esse valor de atributo.

18. search_number:

Pesquisa por monstros em um arquivo de monstros ('monsters.txt') com base em uma categoria de atributo escolhida pelo usuário. Para cada categoria, exibe os monstros que correspondem à categoria de atributo selecionada, usando os índices fornecidos no dicionário. Exibe também a porcentagem de monstros que se encaixam na categoria em relação ao total de monstros.

Args: file (str): O nome do arquivo que contém os dados dos monstros (deve estar no formato 'monsters.txt')

string (str): A string que descreve o tipo de atributo (por exemplo, "tamanho", "tipo", etc.).

dictionary (dict): Um dicionário onde as chaves são as categorias de atributos (como "Tamanho", "Tipo", etc.) e os valores são listas de índices correspondentes às linhas dos monstros no arquivo 'monsters.txt'.

19. search_string:

Pesquisa por monstros em um arquivo de monstros ('monsters.txt') com base em uma categoria de atributo escolhida pelo usuário. Para cada categoria, exibe os monstros que correspondem à categoria de atributo selecionada, usando os índices fornecidos no dicionário. Exibe também a porcentagem de monstros que se encaixam na categoria em relação ao total de monstros.

Args: file (str): O nome do arquivo que contém os dados dos monstros (deve estar no formato 'monsters.txt').

string (str): A string que descreve o tipo de atributo (por exemplo, "AC", "HP", etc.).

dictionary (dict): Um dicionário onde as chaves são as categorias de atributos (como "AC", "HP", etc.) e os valores são listas de índices correspondentes às linhas dos monstros no arquivo 'monsters.txt'.

20. search_name:

Solicita ao usuário o nome de um monstro, divide o nome em palavras-chave e pesquisa essas palavras no índice de nomes para encontrar monstros correspondentes. Em seguida, exibe os monstros encontrados no arquivo de monstros ('monsters.txt'). A pesquisa é feita de forma que todos os termos fornecidos pelo usuário devem estar presentes no nome do monstro para que ele seja considerado uma correspondência. Se um ou mais monstros corresponderem a pesquisa, eles serão exibidos. Caso contrário, uma mensagem informando que o monstro não foi encontrado será mostrada. Também é exibido o total de monstros encontrados e a porcentagem desses monstros em relação ao total de monstros no arquivo.

21. menu_search:

Exibe um menu com opções de atributos para pesquisa e solicita ao usuário que escolha um tipo de pesquisa a ser realizada. O menu inclui as opções de pesquisa por nome, tamanho, tipo, alinhamento, CA, HP, CR e fonte. Após exibir as opções, a função aguarda a escolha do usuário e valida a entrada para garantir que ela esteja entre as opções válidas (1 a 8). Caso a entrada seja invalidada, o usuário será solicitado a inserir novamente.

Returns: int: O número correspondente à opção escolhida pelo usuário (1 a 8).

22. search:

Permite ao usuário realizar pesquisas de monstros com base em diferentes atributos. O usuário escolhe o tipo de pesquisa através de um menu interativo, e a função chama a função correspondente para realizar a pesquisa de acordo com a escolha. O processo é repetido enquanto o usuário desejar realizar mais pesquisas. O menu oferece as opções de pesquisa por nome, tamanho, tipo, alinhamento, CA, HP, CR e fonte.

O fluxo da função é o seguinte:

1. Exibe um menu de opções de pesquisa e coleta a escolha do usuário.
2. Dependendo da escolha do usuário, a função chama a função apropriada (como `search\_name`, `search\_string`, `search\_number`).
3. Após cada pesquisa, a função pergunta ao usuário se ele deseja realizar outra pesquisa.
4. Se o usuário desejar continuar, o loop é reiniciado. Caso contrário, a pesquisa termina.

23. main:

Controla o fluxo principal do programa, interagindo com o usuário para executar ações como carregar dados, adicionar monstros, realizar pesquisas ou sair do programa.

O programa oferece as seguintes opções para o usuário:

1. Recarregar o arquivo 'Monsters_D&D5e.csv', carregando os dados e criando índices.
2. Carregar mais monstros, adicionando-os ao arquivo e criando novos índices.
3. Realizar uma pesquisa de monstros com base em atributos específicos.
4. Sair do programa.

O fluxo de operação depende da escolha do usuário, e a função continua a executar até que o usuário escolha sair.

4.2 tress.py

1. class TrieNode:

- 1.1 __init__:

Inicializa um nó de Trie.

- `children`: dicionário de filhos do nó, onde a chave é o caractere e o valor é o próximo nó.
- `is\_end\_of\_word`: booleano que indica se este nó é o fim de uma palavra.
- `indexes`: uma string que armazena os índices dos monstros associados ao atributo.

2. class Trie:

- 2.1 __init__:

Inicializa a Trie com a raiz como um TrieNode vazio.

2.2 insert:

Insere um atributo de monstro na Trie. Para cada caractere no atributo do monstro, insere um nó correspondente na Trie. Se o nó já existir, move para ele, caso contrário, cria um novo nó. Quando o fim da palavra é alcançado, marca o nó como `'is_end_of_word'` e armazena o índice.

Args: monster (Monster_Attribute): O objeto Monster_Attribute que contém o atributo a ser inserido.

2.3 in_order:

Realiza uma busca em ordem (in-order transversal) na Trie e retorna todos os atributos com seus índices.

Returns: list: Lista de tuplas (atributo, índices), em ordem alfabética.

2.4 _dfs:

Função auxiliar para realizar uma busca em profundidade (DFS) na Trie.

Args: node (TrieNode): O nó atual da Trie.

prefix (str): O prefixo formado até o momento.

attributes (list): A lista que será preenchida com os resultados.

3. class BTSNode:

3.1 __init__:

Inicializa um nó de árvore binária com um chave (atributo).

Args: key (Monster_Attribute): O atributo que será armazenado no nó.

4. class BST:

4.1 __init__:

Inicializa a árvore binária com a raiz como `None`.

4.2 insert:

Insere um novo atributo de monstro na árvore binária. A inserção segue a lógica de árvore binária de busca, comparando o atributo e colocando-o à esquerda ou à direita. Se o atributo já existir, os índices são adicionados ao nó correspondente.

Args: monster (Monster_Attribute): O objeto Monster_Attribute a ser inserido na árvore.

4.3 _insert:

Função recursiva que realiza a inserção do novo atributo na árvore binária.

Args: node (BSTNode): O na atual da árvore.

new_attribute (Monster_Attribute): O novo atributo a ser inserido.

4.4 in_order:

Realiza a busca em ordem (in-order transversal) na árvore binária e retorna os atributos com seus índices.

Returns: list: Lista de tuplas (atributo, índices), em ordem crescente.

4.5 _in_order:

Função auxiliar para realizar a busca em ordem na árvore binária.

Args: node (BSTNode): O na atual da árvore.

result (list): A lista que será preenchida com os resultados.

4.3 monsters.py

1. class Monster:

1.1 __init__:

Inicializa um monstro com diversos atributos.

Atributos:

name (str): O nome do monstro, truncado para 32 caracteres.

size (str): O tamanho do monstro, truncado para 10 caracteres.

type (str): O tipo do monstro, truncado para 40 caracteres.

alignment (str): O alinhamento do monstro, truncado para 11 caracteres.

AC: A classe de armadura (Armor Class) do monstro (pode ser str ou int).

HP: Os pontos de vida (Hit Points) do monstro (pode ser str ou int).

CR: O nível de desafio (Challenge Rating) do monstro (pode ser str ou int).

source (str): A fonte onde o monstro é encontrado, truncada para 30 caracteres.

2. class Monster_Name:

2.1 __init__:

Inicializa o nome de um monstro e seu índice associado.

Atributos:

attribute (str): O nome do monstro, truncado para 32 caracteres.

index (int): O índice associado ao monstro.

3. class Monster_Attribute:

3.1 `__init__`:

Inicializa o nome de um monstro e seu índice associado.

Atributos:

`attribute (str)`: O nome do monstro, truncado para 32 caracteres.

`index (int)`: O índice associado ao monstro.

3.2 `get_index_string`:

Converte a lista de índices em uma string separada por vírgulas.

Retorna: `str`: Uma string contendo todos os índices separados por vírgulas.

5. CONCLUSÃO

Durante o desenvolvimento deste trabalho percebi uma melhora significativa nas minhas habilidades de programação, principalmente no que tange à debugação. Além disso, aprendi a visualizar dados sob várias perspectivas ao mesmo tempo.

Optei por utilizar python uma vez que ela possui uma gramática mais simplificada do que C e me sinto mais confortável programando nela. Sei, porém, que o python possui nativamente algumas estruturas de dados primitivas, como listas, tuplas e dicionários. Durante o desenvolvimento do trabalho, optei por usá-los em alguns momentos dentro de funções para guardar dados que serão posteriormente impressos ao usuário ou transferidos para alguma estrutura de dados própria da aplicação. Também acho interessante pontuar que essas estruturas também podem ser construídas em C, mas acabam demandando mais atenção do programador ao utilizá-las. As listas podem ser implementadas usando listas encadeadas,

os dicionários com uma mistura do tipo de dado da chave e uma lista encadeada e a tupla com uma estrutura com dois campos

Para elaboração do trabalho utilizei 3 módulos do python: collections, csv e typing. O único que introduz uma nova estrutura de dados é o primeiro, de onde usei a estrutura *defaultdict*, um dicionário que guarda a ordem que os valores são adicionados. Essa estrutura foi usada em uma função no trabalho, a função `load_index`, que utiliza essa estrutura pois, ao tentar ligar um valor a uma chave inexistente, ela cria a chave ao invés de dar erro. Para contornar isso, bastaria inserir uma verificação antes da inserção, que, caso indique que a chave não existe, a cria e depois prossegue. Já o módulo csv foi utilizado para ler o arquivo .cvs que contém os dados brutos e o módulo typing foi usado somente para a documentação do código, e não em nenhuma função em si.

Sobre desafios em relação a implementação, acho necessário pontuar as dificuldades obtidas em relação ao uso de arquivo binário. Inicialmente, comecei a elaborar as funções mexendo diretamente em arquivos binários e convertendo os dados para utf-8 apenas no instante de imprimi-los na tela. Contudo, rapidamente notei que o programa não estava funcionando da maneira prevista, provavelmente pela minha falta de experiência utilizando-os. Resolvi então elaborar o programa como se fosse utilizar arquivos texto e, toda vez que alguma informação de um arquivo se mostrasse necessária, convertê-lo inteiramente do binário. Quando fui começar a implementar a gravação e leitura de arquivos binários nessa estratégia notei que o programa estava muito mais lento do que com os arquivos texto. Logo, resolvi consultar o professor orientador, que pediu uma explicação para a aparente inversão da eficiência temporal entre os diferentes tipos de arquivo. Não tenho absoluta certeza, mas imagino que a conversão de todo o arquivo tome mais tempo do que é ganho consultando por meio do arquivo binário em si. Junto com o código-fonte do trabalho terminado mandei

também uma pasta com uma versão anterior do trabalho utilizando arquivos binários, bastante desatualizada em relação ao produto final.